

Лабораторная работа №7

Основы информационной безопасности

Краснова К. Г., НКАбд-03-24

17 февраля 2026

Российский университет дружбы народов, Москва, Россия

Освоить на практике применение режима однократного гаммирования

Нужно подобрать ключ, чтобы получить сообщение «С Новым Годом, друзья!». Требуется разработать приложение, позволяющее шифровать и дешифровать данные в режиме однократного гаммирования. Приложение должно:

1. Определить вид шифротекста при известном ключе и известном открытом тексте.
2. Определить ключ, с помощью которого шифротекст может быть преобразован в некоторый фрагмент текста, представляющий собой один из возможных вариантов прочтения открытого текста

Предложенная Г. С. Вернамом так называемая «схема однократного использования (гаммирования)» является простой, но надёжной схемой шифрования данных. (course?)

Гаммирование представляет собой наложение (снятие) на открытые (зашифрованные) данные последовательности элементов других данных, полученной с помощью некоторого криптографического алгоритма, для получения зашифрованных (открытых) данных. Иными словами, наложение гаммы — это сложение её элементов с элементами открытого (закрытого) текста по некоторому фиксированному модулю, значение которого представляет собой известную часть алгоритма шифрования.

В соответствии с теорией криптоанализа, если в методе шифрования используется однократная вероятностная гамма (однократное гаммирование) той же длины, что и подлежащий сокрытию текст, то текст нельзя раскрыть. Даже при раскрытии части последовательности гаммы нельзя получить информацию о всём скрываемом тексте.

Наложение гаммы по сути представляет собой выполнение операции сложения по модулю 2 (XOR) (обозначаемая знаком $\oplus$) между элементами гаммы и элементами подлежащего сокрытию текста. Напомним, как работает операция XOR над битами: $0 \oplus 0 = 0$, $0 \oplus 1 = 1$, $1 \oplus 0 = 1$, $1 \oplus 1 = 0$.

Такой метод шифрования является симметричным, так как двойное прибавление одной и той же величины по модулю 2 восстанавливает исходное значение, а шифрование и расшифрование выполняется одной и той же программой.

Если известны ключ и открытый текст, то задача нахождения шифротекста заключается в применении к каждому символу открытого текста следующего правила:

$$C_i = P_i \oplus K_i, (7.1)$$

где C_i — i -й символ получившегося зашифрованного послания, P_i — i -й символ открытого текста, K_i — i -й символ ключа, $i = 1, m$. Размерности открытого текста и ключа должны совпадать, и полученный шифротекст будет такой же длины.

Если известны шифротекст и открытый текст, то задача нахождения ключа решается также в соответствии с (7.1), а именно, обе части равенства необходимо сложить по модулю 2 с P_i :

$$C_i \oplus P_i = P_i \oplus K_i \oplus P_i = K_i,$$

$$K_i = C_i \oplus P_i.$$

Открытый текст имеет символьный вид, а ключ — шестнадцатеричное представление. Ключ также можно представить в символьном виде, воспользовавшись таблицей ASCII-кодов.

К. Шеннон доказал абсолютную стойкость шифра в случае, когда однократно используемый ключ, длиной, равной длине исходного сообщения, является фрагментом истинно случайной двоичной последовательности с равномерным законом распределения. Криптоалгоритм не даёт никакой информации об открытом тексте: при известном зашифрованном сообщении C все различные ключевые последовательности K возможны и равновероятны, а значит, возможны и любые сообщения P .

Необходимые и достаточные условия абсолютной стойкости шифра:

- полная случайность ключа;
- равенство длин ключа и открытого текста;
- однократное использование ключа.

Рассмотрим пример.

Ключ Центра:

05 0C 17 7F 0E 4E 37 D2 94 10 09 2E 22 57 FF C8 0B B2 70 54

Сообщение Центра:

Штирлиц – Вы Герой!!

D8 F2 E8 F0 EB E8 F6 20 2D 20 C2 FB 20 C3 E5 F0 EE E9 21 21

Зашифрованный текст, находящийся у Мюллера:

DD FE FF 8F E5 A6 C1 F2 B9 30 CB D5 02 94 1A 38 E5 5B 51 75

Дешифровальщики попробовали ключ:

05 0C 17 7F 0E 4E 37 D2 94 10 09 2E 22 55 E4 D3 07 BB BC 54

Другие ключи дадут лишь новые фразы, пословицы, стихотворные строфы, словом, всевозможные тексты заданной длины.

Выполнение лабораторной работы

Я выполняла лабораторную работу на языке программирования Python, листинг программы и результаты выполнения приведены в отчете.

Требуется разработать программу, позволяющее шифровать и дешифровать данные в режиме однократного гаммирования. Начнем с создания функции для генерации случайного ключа (рис. (fig:001?)).

```
import random
import string

def key(s): 1 usage
    key = ''
    for i in range(len(s)):
        key += random.choice(string.ascii_letters + string.digits)
    return key
```

Рис. 1: Функция генерации ключа

Необходимо определить вид шифротекста при известном ключе и известном открытом тексте. Так как операция исключающего или отменяет сама себя, делаю одну функцию и для шифрования и для дешифрования текста (рис. (fig:002?)).

```
def crypt(s, key): 3 usages
    new_text = ''
    for i in range(len(s)):
        new_text += chr(ord(s[i]) ^ ord(key[i % len(key)]))
    return new_text
```

Рис. 2: Функция для шифрования текста

Выполнение лабораторной работы

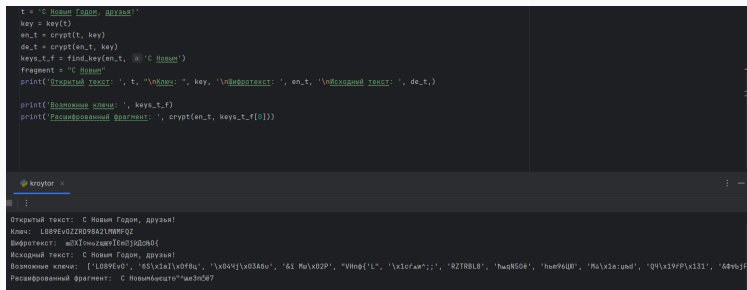
Нужно определить ключ, с помощью которого шифротекст может быть преобразован в некоторый фрагмент текста, представляющий собой один из возможных вариантов прочтения открытого текста. Для этого создаю функцию для нахождения возможных ключей для фрагмента текста (рис. (fig:003?)).

```
def find_key(s, a):  
    possible_keys = []  
    for i in range(len(s) - len(a) + 1):  
        possible_key = ""  
        for j in range(len(a)):  
            possible_key += chr(ord(s[i + j]) ^ ord(a[j]))  
        possible_keys.append(possible_key)  
    return possible_keys
```

Рис. 3: Подбор возможных ключей для фрагмента

Выполнение лабораторной работы

Проверка работы всех функций. Шифрование и дешифрование происходит верно, как и нахождение ключей, с помощью которых можно расшифровать верно только кусок текста (рис. (fig:004?)).



```
t = "С Новым Годом, друзья!"
key = key(t)
en_t = crypt(t, key)
de_t = crypt(en_t, key)
keys_t_f = find_key(en_t, "С Новым")
fragment = "С Новым"
print("Открытый текст: ", t, "\nКлюч: ", key, "\nШифротекст: ", en_t, "\nИсходный текст: ", de_t, )

print("Возможные ключи: ", keys_t_f)
print("Расшифрованный фрагмент: ", crypt(en_t, keys_t_f[0]))
```

Output:

```
Открытый текст: С Новым Годом, друзья!
Ключ: L089Ev0ZZR0V8A2LHMFQZ
Шифротекст: m0Xl0m0z0m0Yl0m0Jl0m0Q{
Исходный текст: С Новым Годом, друзья!
Возможные ключи: ['L089Ev0', '6Sx10l1x0f0u', '\x04cj\x03A0u', '6i M\x02P', "VHnp'L", '\x1cfAm;', 'R2TRBL8', 'hmqNS06', 'hmq96U0', 'M0\x1e:u0d', 'Q4\x19fP\x131', '6Fv0jF']
Расшифрованный фрагмент: С НовымГодом"m03m067
```

Рис. 4: Результат работы программы

Выводы

В ходе выполнения данной лабораторной работы мной было освоено на практике применение режима однократного гаммирования.